Preventing Wrong-Target Agent Actions in Spatial Interfaces with Executable Interaction Contracts

ANONYMOUS AUTHOR(S)

Embodied agents in three-dimensional interfaces can return a fluent response even when the selected object, responsible agent, invoked operation, and reported evidence do not agree. Such failures are difficult to detect when scene bindings, agent roles, handoffs, and backend endpoints are wired in separate artifacts. We present an executable interaction contract for spatial multi-agent interfaces, realized as FunctionalMLDS across a Unity desktop client and a conversational-agent backend. The contract pins one model version and requires each admitted request to resolve one spatial target, one declared provider, one capability use, and one concrete runtime action. The backend re-resolves these relations rather than trusting client fields; stale, ambiguous, unreachable, or multiply bound requests fail before a response-model call and session mutation whenever the violation is request-decidable. Response-dependent violations are rejected transactionally.

We evaluate coverage, migration non-regression, runtime enforcement, added cross-layer expressiveness, and structural cost in three authored spatial environments. All 93 scene assets participate in complete declared interaction paths (186/186 asset-targeting step–capability-use pairs). A normalized direct-wiring adapter and the contract representation agree on all 93 owner decisions and 459 object-by-start-agent route classifications under a fixed shared policy. The executable backend admits all 298 local or directly reachable routes with matching target and provider evidence, and rejects all 161 transitive or unreachable routes before the deterministic response stub without retained session mutation. A stricter provider contract detects and localizes 9/9 additional cross-layer mutations with 0/3 false positives; the direct artifacts do not represent the corresponding typed relations. This additional structure costs 5.9–7.6 times the local adapter workload in the reported microbenchmark. The contribution is a fail-closed control and evidence boundary for an intelligent spatial interface, not a claim about answer quality or human experience.

CCS Concepts: • **Human-centered computing** → **Intelligent user interfaces**; *Interactive systems and tools*; • **Computing methodologies** → *Intelligent agents.*

Additional Key Words and Phrases: intelligent user interfaces, embodied agents, spatial grounding, interaction contracts, runtime assurance, model-driven engineering, Unity

ACM Reference Format:

Anonymous Author(s). 2026. Preventing Wrong-Target Agent Actions in Spatial Interfaces with Executable Interaction Contracts. 1, 1 (August 2026), 22 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

A visitor in a virtual exhibition points at a product panel and asks an embodied agent, “What is this?” The expected interaction appears simple: preserve the visible selection, contact an agent that is responsible for the selected object, invoke the grounded-answer operation, and present a response that still refers to the same object. Yet those decisions often live in different parts of an implementation. Unity stores colliders and scene objects; configuration files assign

Manuscript submitted to ACM

knowledge and roles; orchestration code selects an agent; an endpoint invokes a language model; and logs record only that a request succeeded. Each part can be locally valid while the composed interaction is wrong.

This mismatch is particularly consequential in an intelligent interface. A language model can produce a plausible sentence even when the interface has lost the user's target, routed to an unrelated agent, or invoked an operation that was never declared for that target. Transport success therefore does not establish interaction success. The interface needs a boundary that answers four deterministic questions before probabilistic generation begins: *Which modeled entity was selected? Which agent is responsible? Which capability may be used? Which concrete action realizes it?* After generation, the same boundary must determine whether the returned evidence still agrees with those decisions.

We investigate an *executable interaction contract* for this purpose. The contract connects a user-facing scenario and spatial target to a provider, capability use, runtime binding, runtime action, expected assertion, and observation. Its current realization, FunctionalMLDS (Functional Meta-Language-defined Structure), extends an MLDS-style machine-readable artifact with functional interaction relations [15]. A generation pipeline materializes the contract for a Unity desktop client and a multi-agent backend. The client provides a ray-selected target candidate, but the backend treats the pinned model as authoritative: it validates the candidate's stable identity, derives its group and zone, resolves one responsible provider, checks a permitted one-hop route, and selects one target- and provider-specific runtime binding. No endpoint name supplied by the client can substitute for that path.

We call a request *contract-grounded* when its target, provider, capability use, capability, binding, action, and expected assertion resolve within one pinned model version. The guarantee is conditional but concrete. For an admitted request, the implemented preflight found one complete, model-local path and the response evidence agrees with the trusted target and provider. A request-decidable violation stops before a response-model call or session mutation. A response-dependent violation, such as a generated handoff to an undeclared destination, causes transactional rollback. In both cases the system can report the enforcement stage and violated relation rather than a generic chat failure.

This guarantee deliberately begins *after* perceptual target selection. It does not infer an unobserved intention, prove that ray geometry is honest, or establish factual correctness of generated language. A multimodal reference resolver can propose the candidate; the contract determines whether that candidate has an admissible path through the interface. The resulting separation is useful because reference inference and action permission fail in different ways and require different evidence.

The work is motivated by three interaction requirements drawn from the implemented visitor path and established human–AI interaction guidance [1, 35, 36]:

Stable reference. The object highlighted by the client remains the target while the visitor composes the request, and the response is presented only if the returned identifiers match that pending selection.

Visible unresolved states. No target and ambiguous target are explicit states. They block a deictic request rather than falling through to a fluent ungrounded answer.

Inspectable responsibility. The chosen provider, route reason, capability, binding, and action are retained as structured evidence for developer diagnosis.

We do not infer that these mechanisms improve trust, preference, or usability; that requires human evaluation. We instead ask whether the system implements the claimed control boundary, whether migration preserves the behavior already encoded in the direct artifacts, and what assurance and cost the additional relations introduce.

Accordingly, we study three research questions:

RQ1—Coverage and non-regression. Can the authored spatial cases be migrated to complete target–provider–capability–action paths while preserving their existing ownership and routing decisions under a fixed policy?

RQ2—Fail-closed enforcement. Does the implemented Unity–backend path admit exactly the contract-valid local and one-hop requests, preserve target and provider evidence, and reject stale, ambiguous, unreachable, or contradictory requests without retained side effects?

RQ3—Added assurance and cost. Which cross-layer inconsistencies are expressible and localizable in the contract representation beyond the direct artifacts, and what representation and local execution costs accompany that structure?

The paper makes four contributions:

(1) an interaction-oriented failure model and executable contract that binds a spatial selection to one provider, capability, action, and evidence path under a pinned model version;
(2) an implemented fail-closed architecture across Unity and a conversational-agent backend, including authoritative server-side re-resolution, staged admission checks, one-hop handoff constraints, and transactional response validation;
(3) a reproducible, API-free evaluation over three spatial environments, comprising complete-chain audits, a normalized direct-wiring non-regression comparison, 459 executable routing probes, targeted invalid requests, and Unity batch smokes; and
(4) a separated analysis of common faults, contract-only cross-layer faults, representation edits, and local structural runtime, exposing both the added invariant scope and its cost.

The central claim is not that a metamodel by itself makes an interface intelligent. Rather, the explicit relations determine what the intelligent interface is allowed to interpret and present. This shifts a class of wrong-target, wrong-provider, and undeclared-action failures out of probabilistic dialogue behavior and into a deterministic, inspectable boundary.

2 Related Work and Positioning

2.1 Embodied agents and spatial reference

Embodied conversational agents combine language with a visible body, gesture, gaze, and shared space [10]. Museum-guide systems show why the location and social role of an agent matter to interaction [3, 21], while recent generative-agent and multi-agent work adds memory, role specialization, and language-mediated coordination [25, 32]. These systems motivate the multi-agent setting considered here, but a role or dialogue policy does not by itself bind an utterance to one Unity entity and one executable operation.

Situated-language research addresses the upstream reference problem. Approaches combine plan recognition with referring expressions [16], map directions into perceived environments [20], maintain symbolic anchors for discourse [23], and learn perceptually grounded word meanings [19]. MM-Conv extends this line with synchronized speech, motion, gaze, and scene context for ambiguous reference in dynamic 3D dialogue [13]. Our contract does not compete with these resolvers: it accepts a candidate selection and checks whether the modeled interface permits that target to reach a specific provider and action. A learned or multimodal resolver could therefore replace the current desktop ray without changing the downstream admission questions.

2.2 Conversational XR and model-based interface engineering

Language models increasingly author or operate immersive environments. LLMR uses prompting, scene understanding, planning, execution, and self-debugging to edit mixed-reality worlds [12]. Tell-XR supports conversational end-user development of event–condition–action automations [9]; CUIfy packages LLM-powered conversational agents for XR [4]; and AgentHands studies gestures for spatially grounded agent conversations [28]. These systems establish that conversational generation, agent embodiment, and XR interaction are valuable and feasible. The issue studied here is narrower: after a world and its agents exist, can the interface prevent a selected object, responsible agent, backend action, and returned evidence from silently diverging?

Model-based interface development has long separated domain, task, abstract interface, and concrete implementation concerns [7, 34]. UsiXML and MARIA support multiple development paths and abstraction levels [27, 33], while OpenUIDL addresses runtime omni-channel interfaces [31]. Model-driven work for immersive applications likewise generates AR applications from domain models [8] or structures VR requirements around roles, scenes, actions, states, and validation [18]. FunctionalMLDS is not a general UI description language. It specializes the runtime link among spatial entities, embodied-agent responsibility, permitted capabilities, concrete operations, and observable assertions.

The closest lineage is MLDS. Fischer et al. define a Meta-Language-defined Structure as a machine-readable artifact governed by a domain-specific language specification [15]. Prior work already uses this idea for natural-language-to-Unity materialization and optional placement refinement [6], and modular MLDS instructions add profiles, dependencies, versions, and gatekeeper validation for evolving toolchains [5]. Consequently, this paper does not claim novelty for MLDS-to-Unity generation, placement, or modular language specification. Its contribution begins at interaction time: the same typed relations used during materialization remain active as an admission and evidence contract.

2.3 Human–AI control, contracts, and provenance

Human–AI interaction guidelines emphasize communicating system capability and status, making uncertainty visible, enabling correction, and supporting recovery [1, 35, 36]. Explanations should answer questions people actually have [26], but explanation alone does not guarantee improved joint performance [2]. These findings motivate explicit selection states and structured route evidence. They do not justify a claim that the current diagnostic payload is usable or beneficial; the evaluation tests conformance to the declared behavior, not human outcomes.

Design by Contract separates preconditions, postconditions, and invariants [30]; runtime verification evaluates specified properties against execution observations [24]. We adopt this enforcement intuition without claiming formal program verification. The relevant obligations cross implementation layers: a request must use a pinned model, a unique target, a responsible and reachable provider, and an exact action binding; a response must preserve those identifiers. W3C PROV shows how entities, activities, and agents can support exchangeable provenance [22], and failure-attribution work demonstrates that output-only traces are insufficient for multi-agent diagnosis [11]. Our evidence is intentionally smaller than a full reasoning trace: it records contract identifiers and observable outcomes, not private chain-of-thought or an explanation of semantic answer quality.

Requirements traceability is relevant because a link is valuable only when it supports reasoning about change and failure [17, 29]. The evaluation therefore does not count links alone. It tests whether valid cases resolve end to end, whether controlled mutations are localized, whether runtime requests are admitted or rejected at the intended boundary, and what representational cost the links introduce.

Table 1. Positioning by primary unit and assurance locus. The distinctions state the focus of this paper; they do not imply that a cited system lacks mechanisms outside its stated contribution.

Work	Primary represented unit	Main interface/runtime emphasis	Distinction of this work
LLMR [12] and Tell-XR [9]	World edits or event–condition–action automations	Conversational generation and end-user authoring	We constrain an already materialized request through target, provider, capability, and action.
Spatial grounding [16, 23]	Referring expression, perception, and candidate referent	Inferring what a person refers to	We begin after candidate selection and test whether the selected entity has a permitted executable path.
Model-based UI engineering [7, 33, 34]	Tasks, domain concepts, and abstract/concrete interfaces	Transformation across abstraction and deployment levels	We specialize the runtime boundary to spatial entities, agents, capabilities, actions, and evidence.
MLDS lineage [5, 15]	Machine-readable artifacts and modular language specifications	Generation, validation, profiles, and toolchain evolution	We keep one typed artifact active during request admission and response presentation.
This work	Pinned target–provider–capability–action–assertion contract	Fail-closed spatial request admission and relation-level evidence	The contribution is the composed boundary and its executable, corpus-bounded evaluation.

2.4 Positioning

Table 1 distinguishes the unit constrained by representative work. Conversational-XR systems primarily generate or control worlds; reference-resolution systems infer a target; model-based UI approaches connect abstract and concrete interfaces; and multi-agent systems coordinate roles. The present contract starts from a selected candidate and constrains the specific path that may cross the Unity–backend boundary.

3 Interaction Contract

3.1 Failure model

The contract targets failures that can remain hidden behind a successful language-model response. Table 2 separates these failures by the relation that becomes inconsistent. The categories are not claims about their prevalence; they define the boundary implemented and tested in this work.

3.2 Contract objects and executable path

Let a contract-grounded interaction be represented by the tuple

$$\mathcal{I} = \langle m, s, t, p, u, c, b, a, q, o \rangle,$$

where m is a pinned model version and content hash, s is a scenario step, t is the spatial target, p is the provider, u is the capability use in that scenario, c is the reusable capability, b is a runtime binding, a is a concrete runtime action, q is the set of resulting assertions, and o is runtime observation evidence. The required declaration path is

$$s \rightarrow u \rightarrow c \rightarrow b \rightarrow a.$$

Table 2. Cross-layer failures addressed by the executable contract.

Failure	Example	Contract decision	Observable consequence
Target identity	A collider reports an unknown entity, an obsolete model hash, or a source identifier shared by several objects.	Reject before routing.	No response-model call; target relation and stage are reported.
Target ambiguity	The client reports several candidates or a non-resolved deictic state.	Reject rather than selecting a candidate.	The interface remains in an unresolved state.
Responsibility drift	The active agent is not the unique owner and has no declared direct handoff to the owner.	Resolve asset, then group, then zone ownership; require one owner at the highest matching tier and at most one direct handoff.	Local/direct routes proceed; transitive or unreachable routes stop.
Action shortcut	A scenario or client attempts to invoke an endpoint without a matching provider-specific capability and runtime binding.	Require an exact scenario-step–capability-use–capability–binding–action path.	Zero or multiple exact mappings fail closed.
Evidence mismatch	A response names a different target, provider, binding, or action from the pending request.	Reject presentation and retain the failed relation.	A transport-level success is not treated as contract success.
Undeclared handoff	The response model proposes an agent that is not an allowed handoff target.	Validate the proposed response before commit.	Provisional history is rolled back.

The scenario step cannot call a directly. The capability use names both p and its targets, and the executable profile requires exactly one capability and provider for the evaluated interaction. Assertions are linked to a validation case that covers the same use case and executable binding.

Spatial responsibility is resolved independently of display names. For target t, the resolver collects candidate agents at three tiers:

$$R(t) = \begin{cases} R_{\text{asset}}(t), & \text{if non-empty,} \\ R_{\text{group}}(t), & \text{otherwise if non-empty,} \\ R_{\text{zone}}(t), & \text{otherwise.} \end{cases}$$

Exactly one agent at the highest active tier is required. More than one is ambiguous; none is unassigned. If the responsible provider is not the active agent, the current online contract permits exactly one declared handoff edge. Longer graph paths remain useful offline for diagnosing reachability but are not traversed within one request.

3.3 Admission and response conditions

A deictic request is admitted only if all of the following hold:

(1) the session is pinned to m, and the supplied model hash equals the current project hash and stored contract fingerprint;
(2) the supplied entity and source identifier resolve to the same unique scene asset in m, with a resolved, non-ambiguous selection state;
(3) the responsibility rule yields one provider p, and p is either the active agent or one permitted direct handoff away;

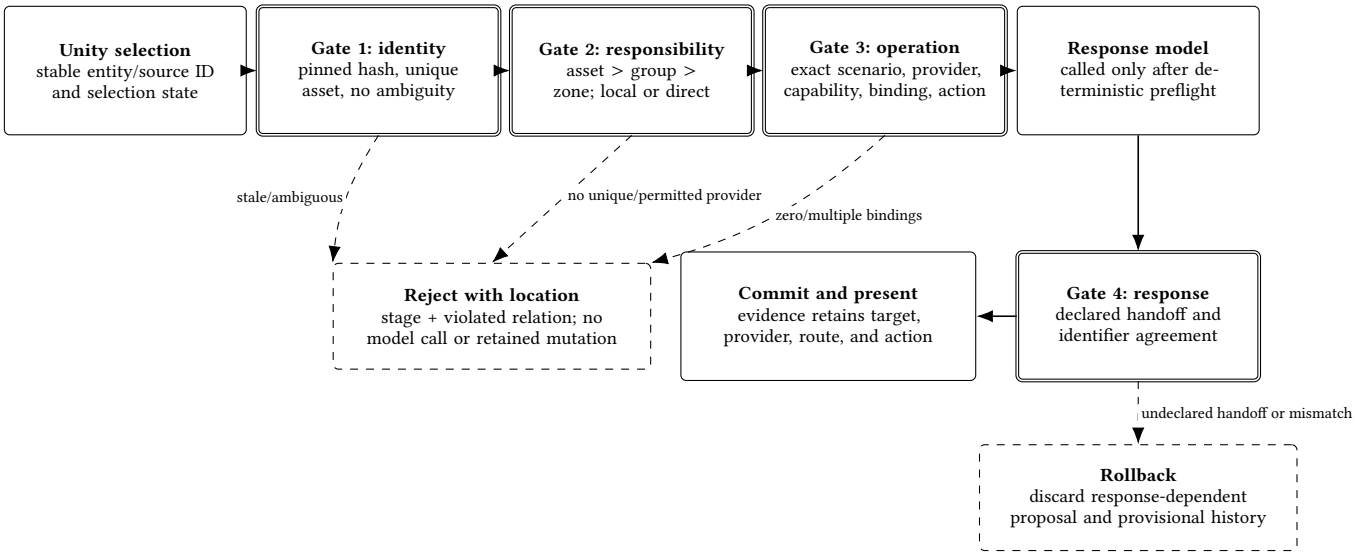

Fig. 1. Fail-closed admission path. Unity proposes a selected target, but the backend re-resolves identity, responsibility, and the exact operation from the pinned contract before generation. Request-decidable failures stop before the response model; response-dependent failures roll back the proposal.

(4) one capability use u for the trusted target and provider resolves to one capability c, binding b, and action a;
(5) the request conforms to the declared action schema.

Conditions one through five are request-decidable and are checked before a response-model call and retained state mutation. The generated proposal is then checked for response-dependent conditions: any proposed handoff must be declared, and the returned model, target, provider, capability, binding, and action identifiers must agree with the preselected path. A violation after generation causes rollback of the provisional conversation state. Later runtime events can evaluate assertions as pass, fail, inconclusive, or error, depending on available observations.

Figure 1 shows the implemented order. The sequence matters: placing a schema check after generation would still allow a stale selection to consume a model call, while trusting a client-supplied provider would let the caller bypass responsibility resolution.

3.4 Guarantee boundary

The implementation provides a *model-consistency guarantee*, not semantic correctness. If a deictic request is admitted, then under the pinned contract it has one trusted asset, one uniquely resolved provider reachable in at most one modeled handoff, and one exact capability-to-action binding; the accepted response carries matching identifiers. This statement does not establish that the person intended the collider selected by the client, that the provider's knowledge is correct, or that generated language answers the question well. Those properties require trusted sensing, semantic ground truth, and human or task-level evaluation outside the present contract.

4 System Realization

4.1 Typed model and executable profile

FunctionalMLDS realizes the contract with a typed object model. Its requirements, use-case, function, and verification concepts reuse a selected EAST-ADL 2.1.12 foundation [14]; the FunctionalMLDS namespace adds scenario flow, entities, embodied agents, capabilities, runtime bindings/actions, assertions, spatial responsibility, and handoffs. EAST-ADL is an implementation substrate for traceability, not the novelty claim, and we do not claim full standard conformance.

Validation is deliberately staged. A JSON schema checks required fields, types, enumerations, and reference shapes. The canonical V2 model checks referential closure, typed cardinalities, unique source identifiers, assertion ownership, and the absence of a direct step-to-action shortcut. The executable pipeline profile tightens compatibility cardinalities where runtime execution needs one decision: a capability use requires one provider and capability, its provider must agree with the performing agent and target responsibility, and a published action must have a complete binding. Runtime checks then bind one specific request and response to an already valid, pinned instance. Keeping these stages separate identifies whether a failure belongs to serialization, language well-formedness, the executable profile, or one runtime observation.

A room model stores stable `id` and `sourceId` values rather than using labels or Unity GameObject names as identity. Agents additionally have a unique source-agent identifier and typed references to grounded assets, object groups, responsible zones, provided capabilities, and handoff targets. Runtime bindings own ordered actions with a locator and optional input/output schemas. A generated runtime projection contains only the data needed for execution, while the canonical instance remains the source of truth.

4.2 Generation and publication

The project pipeline starts from structured room data and frozen semantic artifacts. It normalizes entities, associates objects with groups and zones, derives agent roles and handoffs, materializes local knowledge, computes placement, and assembles scenarios and capability uses. Deterministic stages create stable identifiers, runtime bindings, assertions, validation cases, a trace map, and the backend runtime context. Validators run before an atomic publication step. Content hashes record the model and dependent projections so that stale outputs are detectable.

Language models can propose scene semantics, role labels, handoffs, and knowledge during generation. They do not choose runtime identifiers, decide benchmark routes, or waive structural validation. The frozen evaluation corpus is subsequently regenerated and tested without a network or model API.

4.3 Unity target binding and interaction state

The Unity client imports the materialized room, agent configuration, model projection, and trace map. A scene-object component binds each relevant collider to an exact model entity and source identifier. Scene bootstrap constructs a registry and fails on duplicate entities, duplicate source matches, conflicting manual bindings, or missing target-specific contexts. UUID-like suffixes are not accepted as approximate fallback identifiers.

In the evaluated desktop path, a camera ray selects a registered collider. The client retains the selection while the request is composed and represents `none`, `resolved`, and `ambiguous` as distinct states. A deictic request includes the pinned model hash, entity and source identifiers, candidate identifiers, hit position, distance, selection modality, active agent, and utterance. Numeric ranges, list sizes, and string lengths are bounded. A missing or ambiguous selection is blocked before serialization. The backend remains authoritative even when Unity performs the same earlier check.

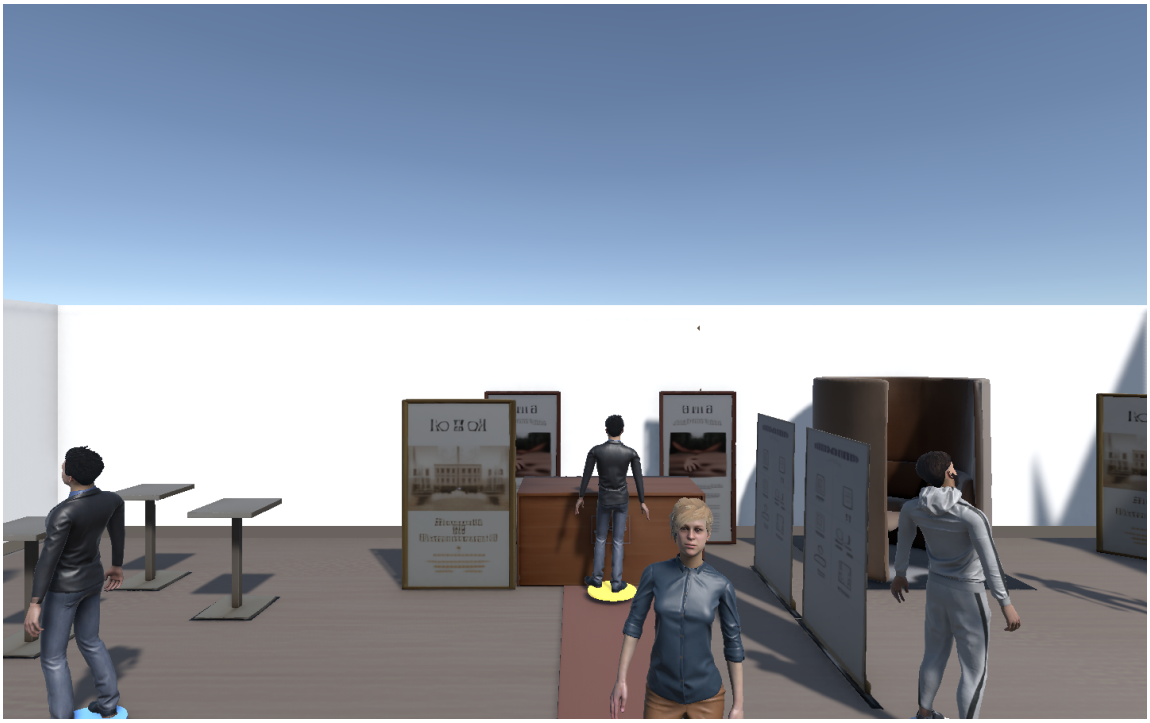

Fig. 2. Implemented Unity desktop view after loading the model-grounded scene and selecting `brand_panel3`. Runtime admission uses the stable entity binding rather than the visible label.

Figure 2 shows the implemented Game view used in the Steinpilz case. The selected source object `brand_panel3` is bound to `ENT-ASSET-BRAND_PANEL3`; the figure is evidence of the desktop interaction path, not a headset study or a polished diagnostic interface.

The multi-scenario bridge does not dispatch solely by an operation kind such as "chat." It first uses the selected entity and server-resolved provider to identify a candidate scenario context. It accepts the scenario, capability, binding, and action identifiers confirmed in the response and forwards the event to the corresponding runner. Zero or multiple exact contexts fail closed.

4.4 Authoritative backend admission

Session setup loads the project contract, validates the native V2 instance and trace map, builds a runtime context, and pins the model hash and contract fingerprint to the session. A changed project cannot be adopted silently: preflight compares the current files with the pinned values and requires a new setup after regeneration.

For each deictic chat request, the backend validates the spatial payload strictly. Unknown fields, non-finite or out-of-range ray values, excessive candidate lists, conflicting entity and source identifiers, unknown objects, non-scene entities, stale hashes, and ambiguous states are rejected. The backend derives the selected asset's group and zone from the runtime context; it does not trust client-supplied ownership fields.

Responsibility resolution applies the asset–group–zone precedence and identifies the source agent's permitted handoff edges. The current request may be answered locally or use one direct declared handoff. Even when a provider

is graph-reachable through several agents, the online request is rejected because one turn does not traverse a transitive path. A later turn can start from the active agent returned by the preceding accepted handoff and is checked again under the same pinned contract.

The backend then selects an exact runtime action using the trusted scenario, provider, and target. A generic endpoint or action-kind match is insufficient. Before generation, it snapshots the mutable session state. If a request-decidable check fails, the response model has not been called. If the structured response proposes an undeclared handoff or fails a postcondition, the snapshot is restored. Only a valid response and its success evidence are committed.

4.5 Evidence and diagnostics

The accepted response and runtime event stream retain the identifiers needed to reconstruct the contract path: model hash, scenario and step, target entity and source object, provider, route reason, capability use, capability, runtime binding, action, and assertion references. Unity compares the returned target, provider, route, and action with its pending selection before presenting the answer. The validation recorder distinguishes a syntactically invoked operation, successful grounding, provider agreement, action agreement, and an observed assertion result. Missing evidence can therefore be `inconclusive` rather than silently passing.

This payload is primarily developer-facing. The visitor sees selection highlighting and unresolved status; route and assertion details are available to logs and runtime components. Designing and evaluating a concise visitor-facing explanation or correction interface remains future work.

4.6 Running example

For `brand_panel3`, the model resolves the source object to `ENT-ASSET-BRAND_PANEL3`, its branding group, and entrance zone. Asset-level responsibility names the Reception Agent. From the Lounge Host, one declared handoff reaches that provider. The target-specific answer step uses the grounded-question capability, which maps through one runtime binding to the backend chat action. The accepted stub response preserves all target, provider, capability-use, capability, binding, and action identifiers. From the Career Advisor, the same provider requires two graph edges, so the request is rejected before the stub and conversational state remains unchanged.

5 Evaluation Method

5.1 Scope and design

We conduct a software-system evaluation of the contract rather than a study of generated language or human experience. The design has four linked parts: (1) a migration and coverage audit over every modeled scene asset, (2) a controlled non-regression comparison with the direct artifacts from which the contract instances were generated, (3) executable backend and Unity contract tests, and (4) mutation and timing analyses that separate common faults from contract-only cross-layer faults. All reported proportions use exact numerators and denominators for the fixed corpus.

The corpus contains the three spatial development cases available in the repository: a career-fair room, a classroom with a dinosaur exhibit, and a food/trade-fair brand room. They contain 27, 36, and 30 routable scene assets and 5, 4, and 6 modeled agents, respectively. The cases were authored during system development, not sampled from a population or held out from implementation decisions. This scope supports exhaustive checks over the three models, not generalization to other rooms or authors.

5.2 RQ1: coverage and non-regression

For each case, the benchmark regenerates a fresh native V2 instance in memory from its frozen version-0.5 case model. Checked-in V2 files are hashed and audited but are not used as treatment input. An asset is complete only if it appears in at least one asset-targeting interaction chain and every such chain resolves exactly one capability and provider, agrees with the scenario performer, contains the evaluated asset among its targets, reaches an executable runtime binding and action, resolves all resulting assertions, and has a use-case-linked validation case covering the binding and assertions. The asset denominator contains every V2 entity with `entityRole=sceneObject`; the chain denominator contains every `ScenarioStep`/`CapabilityUse` pair explicitly targeting one of those assets. A complete chain is a declaration and traceability result, not evidence that a conversation was executed.

The non-regression comparison uses two independently parsed adapters over the same frozen cases:

Direct wiring. Reads the existing scene-semantics, generated agent-role, and handoff-matrix artifacts.
Contract representation. Reads the fresh V2 instance regenerated in the same benchmark run.

Both expose a normalized ownership graph and the same decision policy: asset assignment outranks object-group assignment, which outranks zone responsibility; more than one owner at the highest active tier is ambiguous. For every routeable object, the benchmark compares the owner and then classifies a route from every modeled starting agent as local, directly permitted, transitively graph-reachable, or unreachable.

The parsers extract candidate sets and handoff edges separately, but one shared routing function classifies both normalized views. Agreement is consequently a migration and serialization non-regression check under a fixed policy, not independent evidence that the policy itself is correct.

All natural objects currently have asset-level owners. To exercise the entire precedence rule without misrepresenting corpus frequencies, a separate derived suite copies one object per case and creates three probes: remove its asset owner to require a group fallback; remove both asset and group owners to require a zone fallback; and add a competing owner at the highest active tier to require ambiguity rejection. The direct artifacts have no native group relation, so the benchmark inserts the same normalized group candidate into both copies. These probes test decision behavior, not equivalent native authoring expressiveness.

5.3 RQ2: fail-closed runtime enforcement

The exhaustive runtime sweep converts all object-by-start-agent combinations into backend requests, for 459 probes in total. The expected owner and route class come from the frozen direct-wiring projection. A fresh V2 project is materialized and verified equal to the in-memory treatment before execution. Each probe runs through `SessionStore.chat` with a deterministic structured-response stub and an active network guard.

Local and direct routes are expected to be admitted. An admitted response must preserve the expected target and provider and contain non-empty, cross-field-consistent capability-use, capability, binding, and action identifiers. Transitive and unreachable routes are expected to be rejected because one request permits at most one direct handoff. A rejected request must stop before the stub and leave session state unchanged. The benchmark restores a session snapshot after every probe so that cases remain independent.

A targeted spatial-contract suite exercises one valid deictic request and 16 invalid variants, including missing context, unknown or contradictory identifiers, obsolete hashes, ambiguous candidates, invalid active agents, oversized fields, non-finite coordinates, and an undeclared handoff proposed by the stub. Request-decidable variants require zero stub

calls and no retained mutation. The undeclared handoff is knowable only after generation; it requires one stub call and must then roll back provisional state.

Three Unity Editor batch entry points load the same generated artifacts and runtime components used by the client. They exercise scenario progression, dispatch, five assertion types, valid and broken evidence, desktop-ray selection, ambiguous binding registries, exact target-specific multi-scenario selection, and permission checks. Exit status and explicit success markers are recorded. These are executable Editor paths, not participant or headset trials.

5.4 RQ3: fault scope, representation edits, and local cost

The *common* mutation suite applies three changes that both representations can express, once per case: remove all ownership paths for one object, add a second asset-level owner, and replace a handoff target with a dangling identifier. Each mutation starts from an expected-valid copy. A detection counts only a new issue relative to that copy; localization requires the new issue to name a mutation-specific object, agent, or field token. We record false positives, edited top-level artifacts, and added, removed, or replaced stored references. Edit counts describe deterministic patches, not developer effort.

A separate *contract-only* suite mutates typed relations absent from the direct representation: it removes a capability-use provider, duplicates an agent's source-agent identifier, or assigns a domain capability to the runtime orchestrator. Each mutation is applied once per case. We report the canonical V2 validator and the stricter pipeline agent–provider contract separately. The baseline is marked "not representable" rather than failed; these results measure added invariant scope, not comparative superiority on an impossible task.

For descriptive cost, each adapter repeatedly constructs its normalized responsibility view and enumerates all routes. After four warm-ups, execution order alternates for 40 measured repetitions per case using a monotonic nanosecond clock. The measurement includes adapter construction from already parsed artifacts and route enumeration. It excludes JSON I/O, v0.5-to-V2 generation, Unity, networking, and model latency. We report medians, quartiles, ranges, and the V2/direct median ratio without inferential claims.

5.5 Reproducibility and analysis boundary

The benchmark is API-free and records raw per-probe results, environment metadata, source and input hashes, mutations, validation issues, timing samples, and generated tables. A verifier regenerates fresh V2 instances, reruns targeted tests, checks the three Unity smokes when the Editor is available, and scans the anonymous artifact for identifiers and secrets.

No confidence intervals or significance tests are attached to exhaustive, deterministic probes over three purposively authored cases; doing so would imply a sampling model the design does not have. The evaluation does not measure answer correctness, usefulness, trust, workload, preference, end-to-end latency, or maintenance time. It also does not use a language model as a semantic judge.

6 Results

Table 3 summarizes the principal outcomes. The values are exact for the fixed three-case corpus and deterministic test paths. They are not participant counts or dialogue-quality scores.

6.1 RQ1: complete migration with routing non-regression

All 93 scene assets occurred in at least one declared interaction chain, and all 93 satisfied the asset-completeness definition. The chain audit examined 186 asset-targeting `ScenarioStep`/`CapabilityUse` pairs: 54 in the career-fair

Table 3. Main measured results and their interpretation boundary.

Check	Observed result	Supported interpretation
Complete declarations	93/93 scene assets and 186/186 asset-targeting step–capability-use pairs complete	The three regenerated models contain provider-coherent paths through binding, action, assertion, and validation coverage.
Migration non-regression	93/93 owner decisions and 459/459 route classifications agree	V2 preserves the direct artifacts under the fixed shared resolver; this is not an independent routing oracle.
Runtime admission	298/298 local/direct routes admitted; 161/161 transitive/unreachable routes rejected before the stub	The implemented one-hop policy is enforced and preserves target and provider evidence without retained mutation on rejection.
Common injected faults	Both adapters detected and localized 9/9; 0/3 false positives	Parity for the three shared fault types.
Contract-only faults	Strict provider contract detected and localized 9/9; 0/3 false positives; canonical model validator detected 3/9	The executable profile adds typed cross-layer invariants absent from the direct artifacts.
Unity execution	3/3 batch entry points passed; one multi-scenario smoke loaded 31 contexts and 73 actions and bound 30/30 scene objects exactly	The evaluated desktop runtime components execute the generated contract; no headset or user outcome is established.

case, 72 in the classroom, and 60 in the brand room. Every pair resolved one capability and provider, agreed with its scenario performer and target, reached an executable binding and action, resolved all resulting assertions, and was covered by an appropriate validation case. No asset-chain error was reported. The two pairs per asset encode answer and handoff paths; they are not two executed conversations.

Natural-corpus ownership was unique for 93/93 assets. Every decision occurred at the explicit asset tier; no natural object exercised group or zone fallback, and none was ambiguous or unassigned. In the separate derived priority suite, both adapters passed all nine decision cases: three group fallbacks, three zone fallbacks, and three ambiguity rejections. Across both adapters this is 18/18 expected outcomes, kept separate from natural-corpus frequencies.

The 459 object-by-start-agent routes comprised 93 local-owner cases, 205 direct handoffs, 143 transitive-only graph paths, and 18 unreachable pairs. Local-or-direct one-hop coverage was therefore 298/459 (64.9%), while offline graph reachability was 441/459 (96.1%). The latter does not imply that the online runtime traverses several handoffs within one turn.

The direct-wiring and fresh-V2 adapters selected the same owner for all 93 objects and the same route class for all 459 pairs. They also remained aligned after all nine common mutations. Because both normalized views are classified by one shared policy function, this result establishes migration non-regression and serialization parity, not that V2 discovered a more correct routing policy.

6.2 RQ2: fail-closed enforcement

The executable runtime sweep passed all 459 probes. It admitted 298/298 local or direct routes and made exactly 298 deterministic-stub calls. Every admitted response preserved the expected source object, target entity, and routed provider and returned non-empty capability-use, capability, binding, and action identifiers that agreed across the checked response, grounding, routing, and runtime-action fields.

The runtime rejected 161/161 transitive or unreachable routes before the stub and without retained session mutation. This set contains the 143 transitive-only and 18 unreachable pairs. The result demonstrates the current one-hop

Table 4. Contract-only mutations. "Not represented" means that the direct artifacts contain no equivalent typed relation; it is not counted as a baseline miss.

Mutation (once per scene)	Violated invariant	Direct artifacts	Executable provider contract
Remove `CapabilityUse.provider`	An executable capability occurrence has exactly one provider agreeing with the performing agent and target responsibility.	Not represented	3/3 detected and localized
Duplicate `Agent.sourceAgentId`	Runtime-facing source-agent identity is unique.	No typed equivalent	3/3 detected and localized
Assign a domain capability to the runtime orchestrator	Domain behavior is provided by the modeled domain agent, while the orchestrator realizes infrastructure actions.	Not represented	3/3 detected and localized

admission boundary, not task failure in principle: a system with a different declared policy could permit multi-hop execution, but this implementation does not.

The targeted spatial suite accepted its valid deictic request and rejected all 16 invalid variants without retained mutation. Fifteen variants were request-decidable and stopped before the stub. The remaining variant used a structured response that proposed an undeclared handoff; one stub call was necessary to reveal the destination, after which provisional history was rolled back. Rejections included stale model hashes, unknown or contradictory identifiers, ambiguous selections, invalid agents, excessive payloads, and non-finite or out-of-range spatial values.

All three Unity Editor batch entry points returned success. Together they exercised scenario progression, dispatch, five assertion types, valid and deliberately broken evidence, selection-state blocking, and ambiguous registries. The multi-scenario Steinpilz smoke loaded 31 contexts and 73 actions and established unique exact bindings for 30/30 scene objects without UUID-suffix fallback. These observations confirm that the client-side bridge and evidence components execute the materialized contract; they do not measure frame rate, end-to-end latency, or human understanding.

6.3 RQ3: added invariant scope and cost

On the common mutation denominator, both adapters accepted all three unmodified case copies, for 0/3 false positives per adapter. Both detected and localized 9/9 changes: one missing owner, one duplicate owner, and one dangling handoff per scene. The result is deliberately neutral. Direct wiring can enforce these three relationships when supplied with a validator and the same policy.

The representation patches expose a mixed trade-off. Across the nine common mutations, the contract representation changed 9 top-level artifacts compared with 15 for direct wiring, but changed 18 stored references compared with 15. These counts describe the scripted patches and do not establish that a developer would work faster or make fewer mistakes.

The contract-only suite provides the positive evidence for additional expressiveness. Table 4 reports the three typed mutation classes. The canonical model validator detected the duplicated source-agent identifier in each case (3/9 overall) but intentionally permits some optional relations for compatibility. The executable pipeline agent–provider contract detected and localized all 9/9 mutations with 0/3 false positives on valid copies. The direct artifacts have no typed capability-use provider or domain-capability-to-runtime-orchestrator relation, so there is no equivalent baseline outcome to score.

Table 5. Local structural workload over 40 measured repetitions per adapter and case. Values are median [Q1, Q3] in milliseconds.

Case	Direct	Fresh V2	Ratio
Career fair	1.322 [1.167, 1.516]	10.057 [7.784, 11.573]	7.61
Classroom	1.246 [0.772, 1.609]	9.439 [5.681, 13.336]	7.58
Brand room	1.044 [0.992, 1.694]	6.151 [4.951, 7.309]	5.89

Table 5 reports the current benchmark snapshot. The fresh-V2 adapter required 5.89–7.61 times the direct-wiring median for the measured in-memory workload. Absolute medians ranged from 6.15 to 10.06 ms for V2 and 1.04 to 1.32 ms for direct wiring. Outputs remained deterministic across all repetitions. These values exclude file parsing, generation, Unity, networking, and model inference and must not be read as end-user response latency.

Taken together, RQ3 does not show that an explicit contract is universally better than direct artifacts. It shows a more specific trade: shared ownership and handoff faults can be validated in either representation, while the contract makes additional provider, identity, capability, binding, and evidence relations explicit and enforceable at measurable structural cost.

7 Discussion

7.1 Why the contract is an intelligent-interface contribution

The contract does not generate the linguistic content that makes an agent appear intelligent. It determines whether the interface may use and present that content for a particular spatial interaction. This distinction matters because a wrong-target answer can be both fluent and locally plausible. By binding the pending selection to one provider and action before generation, the system turns a silent semantic-looking failure into a deterministic admission decision.

The user-facing consequence is limited but concrete. A deictic request cannot proceed from a missing or ambiguous selection; a returned answer cannot be presented as grounded when its target or provider disagrees with the pending interaction. The current implementation exposes the detailed route and binding primarily to developers, not visitors. A future interface could use the same evidence to explain, for example, “This object is handled by the Reception Agent” or to offer a clarification choice. Whether such explanations improve understanding, reliance, or recovery remains an empirical question.

7.2 What the direct-wiring comparison establishes

The comparison is intentionally less dramatic than an architecture competition. For explicit asset ownership, handoff edges, and a fixed priority rule, direct artifacts can produce the same routing decisions and detect the same three common fault types. This is an important result: introducing FunctionalMLDS did not change the behavior already encoded in the source artifacts, but neither did the migration automatically improve it.

The added value appears where the interaction crosses artifact boundaries. Direct wiring has no typed occurrence connecting a scenario target to one provider, capability, binding, action, and assertion. Consequently, relations such as “this capability use has no provider” or “this domain capability is provided by the runtime orchestrator” are not representable as baseline invariants. The strict provider contract detected all such injected faults. This supports a scoped adoption argument: the model is useful when a system needs lifecycle-wide target–provider–action evidence, not merely an owner lookup table.

7.3 When the added structure is justified

A static room with one chatbot and a small, stable set of objects may not justify more than direct configuration and tests. The contract becomes more plausible when several agents divide responsibility, the same action kind has many target-specific bindings, scenes are regenerated, or developers need to trace a failure across Unity and backend artifacts. Even then, its cost is not free. The current representation stores more references for the common patches and its normalized adapter is substantially slower than direct wiring, although the measured workload remains small relative to a typical model call.

The benchmark does not measure authoring effort. Fewer changed top-level files could reduce search and coordination, while more explicit references could increase editing burden. A developer study must test this trade rather than inferring it from patch counts.

7.4 Interaction design opportunities

The present boundary supports several interaction patterns that are not yet implemented as evaluated visitor experiences:

Clarification. When several candidates or owners exist, the interface can present the alternatives instead of only blocking the request.

Transparent handoff. Before delegating, the active agent can name the responsible provider and reason, then let the visitor confirm.

Correction. A visitor or author can replace the selected target or report that the proposed responsibility is wrong; the model relation can then be revised through a typed transaction.

Abstention with location. A rejection can distinguish "I cannot identify the object," "no agent is responsible," and "the responsible agent is not reachable" rather than presenting one generic failure.

These patterns use evidence the system already produces, but their wording, timing, and cognitive cost require design and evaluation. They are stronger future directions than adding more internal trace fields without an interface that uses them.

7.5 Priority evidence needed beyond this study

Two additions would most improve external validity. First, a fourth independently authored, frozen scene should contain natural group- and zone-level responsibility, at least one unassigned object, and one genuine ambiguity. Resolver and validator code should be frozen before the case is revealed. This would test transfer without deriving every interesting condition synthetically.

Second, expected routes should be supplied by an independent oracle or a separately implemented resolver rather than the same shared classification function. A declarative table reviewed before execution would be sufficient for a modest held-out case. These technical additions address the strongest circularity concern without making unsupported human claims.

A developer task study is the most relevant human follow-up if ethics review, participants, and sufficient time are available. Participants could diagnose and repair target, provider, and handoff faults using direct artifacts or the contract representation. Completion rate, time, inspected artifacts, incorrect repairs, and explanation accuracy would test the proposed diagnostic benefit. A small convenience study using only subjective ratings would add less evidence than a well-controlled held-out technical evaluation.

7.6 Authorization and policy composition

"Permitted" in this paper means declared by the interaction model. The contract neither authenticates a visitor nor grants access to protected data or actuators. A deployment can first resolve the exact target–provider–action tuple and then submit it to an external authorization policy. Capability lifecycle governance can separately decide which implementation version is active. The current prototype implements neither identity management nor resource authorization and should not be presented as a security boundary by itself.

7.7 Privacy, bias, and accountability

A spatial-agent request can reveal utterances, selected objects, locations, and interaction histories. The backend receives the utterance, stable target identifiers, ray metadata, modality, and bounded history. Persistent runtime events omit the utterance text in the evaluated path, but the prototype lacks visitor-facing consent, retention, export, and deletion controls. A deployment should minimize stored spatial data and separate technical entity identifiers from personal identities.

Explicit agent responsibility can make organizational assumptions easier to inspect, but it can also formalize biased or inappropriate authority. A validator can establish that one provider is declared consistently; it cannot establish that the provider assignment is fair, representative, or safe. Model review, knowledge provenance, refusal behavior, and domain safeguards remain necessary.

8 Limitations and Threats to Validity

The evaluation establishes behavior for three purposively authored development cases. It does not estimate frequencies in a population of rooms, domains, or authors. Every natural asset has an explicit asset-level owner, so group and zone fallback and ambiguity are exercised only in derived copies. No independently authored or held-out scene tests generalization.

The non-regression comparison is controlled rather than independent. Both representations originate from shared semantic artifacts and are classified by one routing function after separate parsing. The direct-wiring projection also supplies expected owner and route classes for the runtime sweep. Agreement therefore detects migration and serialization divergence but cannot validate the shared policy against an external oracle. Binding identifiers are checked for cross-field agreement within one pinned model, not against independently authored binding labels.

Fault injection is systematic but small. The common denominator contains three mutation types, and the contract-only denominator contains three additional types. Detection and localization of these faults do not imply complete fault coverage or automatic repair. Scripted artifact and reference deltas are not developer effort. The V2 assembler, canonical validator, executable provider validator, and materializer share a codebase, so their agreement is not independent standard conformance.

Runtime tests replace the online response model with a deterministic structured stub. This isolates admission, call counts, evidence, and rollback, but it does not measure factual correctness, relevance, naturalness, refusal quality, or the interaction between generated content and the contract. No language model is used as a semantic judge.

The Unity evidence is limited to a desktop ray and Editor batch execution. There is no headset, WebXR controller, gaze, touch, or physical multi-user evaluation. The server validates bounded client-reported ray metadata but cannot prove that the geometry came from a trusted sensor. A deployed system needs authenticated transport and a trusted client boundary.

Sessions pin a static project version. Drift is detected, but the system has no live migration for objects created, deleted, merged, or rebound during a conversation. Online routing permits at most one direct handoff per request; transitive reachability is reported only as an offline diagnostic. Concurrent writes to one session and coordination among simultaneous visitors were not evaluated.

The current visitor interface exposes selection and unresolved status, while the detailed contract evidence is mainly developer-facing. No participant study tests whether people notice or understand the failure states, whether developers diagnose faults faster, or whether added explanations calibrate reliance. We therefore make no claims about usability, trust, workload, preference, immersion, or maintenance advantage.

The local timing benchmark uses one environment and measures only in-memory adapter construction and route enumeration. It excludes parsing, generation, Unity, network, and model latency. Ratios characterize this implementation and snapshot, not end-to-end response time. Finally, the selected EAST-ADL subset serves as a traceability foundation; full EAST-ADL conformance has not been independently assessed.

9 Conclusion

We presented an executable interaction contract that keeps a selected spatial entity, responsible embodied agent, declared capability, concrete backend action, and returned evidence within one pinned model version. The Unity client makes unresolved selection explicit, while the backend re-resolves identity, responsibility, route, and action before generation and rolls back response-dependent violations.

Across three authored environments, all 93 assets and 186 target-specific declaration pairs were complete. Migration preserved all 93 owner decisions and 459 route classifications under a shared policy. The runtime admitted all 298 local or direct routes with matching evidence and rejected all 161 transitive or unreachable routes before the response stub without retained mutation. A strict provider contract additionally detected and localized all nine injected cross-layer faults that rely on relations absent from the direct artifacts, at a measured increase in local structural workload.

These results support a narrow conclusion: an explicit contract can provide a fail-closed control and evidence boundary for the evaluated spatial-agent interface. They do not establish better dialogue or a better user experience. The next decisive evidence is an independently authored held-out scene and, when feasible, a controlled developer study of diagnosis and repair.

GenAI Usage Disclosure

The frozen case-study inputs include eight successful structured-output calls to `gpt-4.1-mini-2025-04-14` at temperature 0.2: two scene-semantics artifacts, three agent-role and handoff artifacts, and three sets of local knowledge entries. A remaining scene-semantics artifact was recovered deterministically without a model call. Prompts, returned JSON, model identifiers, token metadata, and validation records are retained with the case artifacts. The authors reviewed and froze these outputs as study inputs; no generative model supplied evaluation labels or served as a semantic judge. Model assembly, routing probes, fault injection, measurements, and reported tables are deterministic and API-free.

A coding assistant supported prototype hardening, test and analysis code, literature-search support, and drafting and editing. The authors inspected and revised the resulting code and prose, verified citations and generated evidence, and remain responsible for all claims.

A Detailed Contract and Evaluation Tables

A.1 Running-example identifiers

Table 6 spells out the exact objects used for the brand_panel3 walkthrough. Display labels are shortened in the prose, while execution uses these stable identifiers.

Table 6. Exact contract objects for the Steinpilz brand_panel3 interaction.

Element	Exact instance	Runtime meaning
Target	brand_panel3 → ENT-ASSET-BRAND_PANEL3; group ENT-GROUP-BRANDING; zone ENT-ZONE-ENTRANCE_ZONE	Collider observation and trusted model referent
Use case and scenario	UC-STEINPILZ_BRAND_ROOM-INTERACT-BRAND_PANEL3; SC-STEINPILZ_BRAND_ROOM-INTERACT-BRAND_PANEL3-MAIN	User-facing interaction and executable flow
Answer step	STEP-STEINPILZ_BRAND_ROOM-INTERACT-BRAND_PANEL3-ANSWER	Scenario occurrence requiring one capability use
Capability use	CU-STEINPILZ_BRAND_ROOM-INTERACT-BRAND_PANEL3-ANSWER-ROOM-GROUNDED-QUESTION; provider ENT-AGENT-RECEPTION_AGENT	Explicit target tuple and provider
Capability	CAP-STEINPILZ_BRAND_ROOM-ANSWER-ROOM-GROUNDED-QUESTION	Operation declared for the Reception Agent
Binding and action	RB-STEINPILZ_BRAND_ROOM-ANSWER-ROOM-GROUNDED-QUESTION; RA-STEINPILZ_BRAND_ROOM-BACKEND-CHAT	Exact realization at POST /chat
Assertion and case	SA-STEINPILZ_BRAND_ROOM-ANSWER-GROUNDED; VC-STEINPILZ_BRAND_ROOM-INTERACT-BRAND_PANEL3	Declared identifier agreement and validation scope

A.2 Per-case structural results

Table 7. Complete interaction paths. “Pairs” are asset-targeting ScenarioStep/CapabilityUse pairs.

Case	Assets	Complete	Pairs	Complete
Career fair	27	27	54	54
Classroom	36	36	72	72
Brand room	30	30	60	60
Total	93	93	186	186

A.3 Enforcement layers

B Artifact Map

The reproducible benchmark is rooted at research/iui2027/evaluation. Its results.json contains raw records; tables.md contains regenerated paper summaries; and environment.json records hashes and the local timing environment. The verifier and frozen regeneration entry points are under research/iui2027/artifact. Backend runtime code

Table 8. Offline structural routes from every modeled starting agent. "Transitive" and "reachable" are graph properties, not one-turn online execution.

Case	Probes	Local	Direct	Transitive	Unreachable	Reachable
Career fair	135	27	53	55	0	135
Classroom	144	36	81	9	18	126
Brand room	180	30	71	79	0	180
Total	459	93	205	143	18	441

Table 9. Representative invariants and failure boundaries in the implemented interaction path.

Layer	Representative invariant	Failure boundary
JSON schema	Required fields, types, enumerations, reference shapes	Before constructing a model instance
Canonical V2 model	Referential closure, typed cardinalities, source-ID uniqueness, assertion/validation ownership, no direct step-to-action shortcut	Instance rejected before materialization
Executable profile	One provider and capability per occurrence, provider/performer/target agreement, complete binding and action	Invalid chains are not published
Backend runtime	Pinned hash, unique trusted entity, derived group/zone, unique provider, local/direct route, exact target-specific action, request/response schema	Before generation and mutation when request-decidable; rollback after a response-dependent failure
Unity client	Exact collider binding, explicit unresolved states, returned target/provider/route/action agreement	Before request serialization or answer presentation; backend remains authoritative

is under `InteractivAgents/openai_unity_expert_npcs_pycharm/InteractiveAgents/backend`, and Unity runtime components are under the corresponding `unity_scripts/FunctionalMldsV2` directory. The original manuscript remains in `research/iui2027/paper`; this reframed manuscript is deliberately separate.

References

[1] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, 1–13. doi:10.1145/3290605.3300233

[2] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, 1–16. doi:10.1145/3411764.3445717

[3] Timothy Bickmore, Laura Pfeifer, and Daniel Schulman. 2011. Relational Agents Improve Engagement and Learning in Science Museum Visitors. In *Intelligent Virtual Agents*. Springer, 55–67. doi:10.1007/978-3-642-23974-8_7

[4] Kadir Burak Buldu, Süleyman Özdel, Ka Hei Carrie Lau, Mengdi Wang, Daniel Saad, Sofie Schönborn, Auxane Boch, Enkelejda Kasneci, and Efe Bozkir. 2025. CUIfy the XR: An Open-Source Package to Embed LLM-Powered Conversational Agents in XR. In *2025 IEEE International Conference on Artificial Intelligence and eXtended and Virtual Reality*. IEEE, 192–197. doi:10.1109/AIxVR63409.2025.00037

[5] Louis Burk, Alexander Fischer, Uwe Wienkop, Ramin Tavakoli Kolagari, Christoph Scharnagl, and Alexandra Arzberger. 2026. Handling Toolchain Evolution with Modular Meta-Languages: An Approach for Structured LLM-Based Artifact Generation. In *Proceedings of the 21st International Conference on Evaluation of Novel Approaches to Software Engineering*, Vol. 1. SciTePress, 807–814. doi:10.5220/0014715200004015

[6] Louis Burk and Uwe Wienkop. 2025. Dynamische Optimierung von VR-Lernumgebungen durch generative KI. In *Zukunft MINT Lehre: Was bleibt? Was kommt? Was wirkt? Tagungsband zum 6. Symposium zur Hochschullehre in den MINT-Fächern (Didaktiknachrichten)*, Hanna Dölling and Claudia Schäfle (Eds.). BayZiel – Bayerisches Zentrum für Innovative Lehre, München, 300–308. DiNa-Sonderausgabe, ISSN 1612-4537. https://bayziel.de/wp-content/uploads/Tagungsband_MINTSymposium_2025.pdf

[7] Gaëlle Calvary, Joëlle Coutaz, David Thevenin, Quentin Limbourg, Laurent Bouillon, and Jean Vanderdonckt. 2003. A Unifying Reference Framework for Multi-target User Interfaces. *Interacting with Computers* 15, 3 (2003), 289–308. doi:10.1016/S0953-5438(03)00010-9

[8] Rubén Campos-López, Esther Guerra, Juan de Lara, Alessandro Colantoni, and Antonio Garmendia. 2023. Model-Driven Engineering for Augmented Reality. *Journal of Object Technology* 22, 2 (2023), 2:1–2:15. doi:10.5381/jot.2023.22.2.a7

[9] Alessandro Carcangiu, Marco Manca, Jacopo Mereu, Carmen Santoro, Ludovica Simeoli, and Lucio Davide Spano. 2025. Tell-XR: Conversational End-User Development of XR Automations. In *Human-Computer Interaction – INTERACT 2025*. Springer, 617–641. doi:10.1007/978-3-032-04999-5_35

[10] Justine Cassell, Timothy W. Bickmore, Mark Billinghurst, Lee Campbell, Kenny Chang, Hannes Högni Vilhjálmsson, and Hao Yan. 1999. Embodiment in Conversational Interfaces: Rea. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 520–527. doi:10.1145/302979.303150

[11] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. 2026. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 19888–19905. doi:10.18653/v1/2026.acl-long.912

[12] Fernanda De La Torre, Cathy Mengying Fang, Han Huang, Andrzej Banburski-Fahey, Judith Amores Fernandez, and Jaron Lanier. 2024. LLMR: Real-time Prompting of Interactive Worlds using Large Language Models. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 1–22. doi:10.1145/3613904.3642579

[13] Anna Deichler, Jim O'Regan, Fethiye Irmak Dogan, Lubos Marcinek, Anna Klezovich, Iolanda Leite, and Jonas Beskow. 2026. MM-Conv: A Multimodal Dataset and Benchmark for Context-Aware Grounding in 3D Dialogue. arXiv:2605.21796 [cs.CV] doi:10.48550/arXiv.2605.21796

[14] EAST-ADL Association. 2013. *EAST-ADL Domain Model Specification*. Language Specification Version 2.1.12. EAST-ADL Association. https://east-adl.info/Specification/V2.1.12/EAST-ADL-Specification_V2.1.12.pdf

[15] Alexander Fischer, Louis Burk, Ramin Tavakoli Kolagari, and Uwe Wienkop. 2025. Machine-Readable by Design: Language Specifications as the Key to Integrating LLMs into Industrial Tools. In *Proceedings of the 20th Conference on Computer Science and Intelligence Systems*, Vol. 43. Polish Information Processing Society, 531–542. doi:10.15439/2025F5613

[16] Peter Gorniak and Deb Roy. 2005. Probabilistic Grounding of Situated Speech Using Plan Recognition and Reference Resolution. In *Proceedings of the 7th International Conference on Multimodal Interfaces*. ACM, 138–143. doi:10.1145/1088463.1088489

[17] Orlena C. Z. Gotel and Anthony C. W. Finkelstein. 1994. An Analysis of the Requirements Traceability Problem. In *Proceedings of the First IEEE International Conference on Requirements Engineering*. IEEE, 94–101. doi:10.1109/ICRE.1994.292398

[18] Sai Anirudh Karre and Y. Raghu Reddy. 2024. Model-based Approach for Specifying Requirements of Virtual Reality Software Products. *Frontiers in Virtual Reality* 5 (2024), 1471579. doi:10.3389/frvir.2024.1471579

[19] Casey Kennington and David Schlangen. 2015. Simple Learning and Compositional Application of Perceptually Grounded Word Meanings for Incremental Reference Resolution. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 292–301. doi:10.3115/v1/P15-1029

[20] Thomas Kollar, Stefanie Tellex, Deb Roy, and Nicholas Roy. 2010. Toward Understanding Natural Language Directions. In *2010 5th ACM/IEEE International Conference on Human-Robot Interaction*. IEEE, 259–266. doi:10.1109/HRI.2010.5453186

[21] Stefan Kopp, Lars Gesellensetter, Nicole C. Krämer, and Ipke Wachsmuth. 2005. A Conversational Agent as Museum Guide: Design and Evaluation of a Real-World Application. In *Intelligent Virtual Agents*. Springer, 329–343. doi:10.1007/11550617_28

[22] Timothy Lebo, Satya Sahoo, and Deborah McGuinness. 2013. *PROV-O: The PROV Ontology*. W3C Recommendation. World Wide Web Consortium. https://www.w3.org/TR/2013/REC-prov-o-20130430/

[23] Séverin Lemaignan, Raquel Ros, E. Akin Sisbot, Rachid Alami, and Michael Beetz. 2012. Grounding the Interaction: Anchoring Situated Discourse in Everyday Human-Robot Interaction. *International Journal of Social Robotics* 4, 2 (2012), 181–199. doi:10.1007/s12369-011-0123-x

[24] Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *The Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303. doi:10.1016/j.jlap.2008.08.004

[25] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, Vol. 36. 51991–52008. doi:10.52202/075280-2264

[26] Q. Vera Liao, Daniel Gruen, and Sarah Miller. 2020. Questioning the AI: Informing Design Practices for Explainable AI User Experiences. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 1–15. doi:10.1145/3313831.3376590

[27] Quentin Limbourg, Jean Vanderdonckt, Benjamin Michotte, Laurent Bouillon, and Víctor López-Jaquero. 2005. USIXML: A Language Supporting Multi-path Development of User Interfaces. In *Engineering Human Computer Interaction and Interactive Systems*. Springer, 200–220. doi:10.1007/11431879_12

[28] Ziyi Liu, David Li, Zhongyi Zhou, David Kim, Ruofei Du, and Xun Qian. 2026. AgentHands: Generating Interactive Hand Gestures for Spatially Grounded Agent Conversations in XR. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM, 1–24. doi:10.1145/3772318.3790938

[29] Patrick Mäder and Alexander Egyed. 2015. Do Developers Benefit from Requirements Traceability When Evolving and Maintaining a Software System? *Empirical Software Engineering* 20, 2 (2015), 413–441. doi:10.1007/s10664-014-9314-z

[30] Bertrand Meyer. 1992. Applying Design by Contract. *Computer* 25, 10 (1992), 40–51. doi:10.1109/2.161279

[31] Alex Moldovan, Vlad Nicula, Ionut Pasca, Mihai Popa, Jaya Krishna Namburu, Anamaria Oros, and Paul Brie. 2020. OpenUIDL, A User Interface Description Language for Runtime Omni-Channel User Interfaces. *Proceedings of the ACM on Human-Computer Interaction* 4, EICS (2020), 1–52. doi:10.1145/3397874

[32] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 1–22. doi:10.1145/3586183.3606763

[33] Fabio Paternò, Carmen Santoro, and Lucio Davide Spano. 2009. MARIA: A Universal, Declarative, Multiple Abstraction-Level Language for Service-Oriented Applications in Ubiquitous Environments. *ACM Transactions on Computer-Human Interaction* 16, 4 (2009), 1–30. doi:10.1145/1614390.1614394

[34] Angel R. Puerta and Jacob Eisenstein. 1999. Towards a General Computational Framework for Model-Based Interface Development Systems. In *Proceedings of the 4th International Conference on Intelligent User Interfaces*. ACM, 171–178. doi:10.1145/291080.291108

[35] Justin D. Weisz, Jessica He, Michael Muller, Gabriela Hoefer, Rachel Miles, and Werner Geyer. 2024. Design Principles for Generative AI Applications. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 1–22. doi:10.1145/3613904.3642466

[36] Qian Yang, Aaron Steinfeld, Carolyn Rosé, and John Zimmerman. 2020. Re-examining Whether, Why, and How Human-AI Interaction Is Uniquely Difficult to Design. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM, 1–13. doi:10.1145/3313831.3376301